

Платформы и среды выполнения языков программирования

Лекция 1

Булычев Дмитрий dboulytchev@gmail.com

Дмитрий Юрьевич не пользуется нейросетями и в начале пары приводит две красивых аналогии, с этим связанных:

1. Вытеснение нейросетями ручного написания кода похоже на вытеснение калькуляторами логарифмических линеек, вытеснение ручного написания ассемблерного кода компиляторами, вытеснение платформенно-зависимых операционных систем на операционные системы, написанные на языках высокого уровня (например, unix)
2. Если использовать для написания кода только нейросети, то это похоже на то, как если бы спортсмены участвовали только в соревнованиях и совершенно не тренировались между ними. Чтобы хорошо выступить на соревнованиях, необходимо поддерживать себя в форме и тренироваться

ОСНОВЫ

Def. Язык программирования — множество программ.

Nota: Более того — множество представлений программ.

Есть следующие основные составляющие лингвистики, которые мы будем использовать в нашем курсе:

1. Лексика

Лексика изучает слова и знаки. Она отвечает на вопрос: “является ли эта последовательность символов словом языка вообще?” Так, в русском языке нет слов “яаво” и “ьяолв”, но есть слова “ялта” и “парус”, и нельзя использовать китайские иероглифы. “Ялта” — это корректное слово, которое пройдет лексический анализ, а “яаво” — нет. В языках программирования роль слов выполняют токены. Так, например, нельзя начинать имя переменной с цифр — такой токен будет признан некорректным.

2. Синтаксис

Синтаксис изучает запись слов в предложении. Это наши “правила записи программы”. Мы проверяем, могут ли наши слова и символы (токены) стоять в той последовательности, в которой они стоят. Например, у каждой открывающей скобки должна быть ровно одна закрывающая скобка. Синтаксически неверно оставлять скобки незакрытыми, хотя сами скобки и являются корректными символами нашей программы.

3. Семантика

Семантика изучает осмысленность полученных предложений — это “смысл” программы.

Рассмотрим пример:

- “Сядь за стол”
- “Съешь, пожалуйста, этим столом себя”

Оба предложения синтаксически корректны, но последнее, очевидно, вопиющая бессмыслица.

Аналогичный пример есть и в коде:

```
int x = "hello";
```

Синтаксически это корректная инструкция, но семантически она бессмысленна: строковый литерал не может быть значением переменной типа `int`.

Ещё пример — деление на ноль:

```
x = y / 0
```

Синтаксически всё верно, а вот что это означает, зависит от семантики конкретного языка: где-то это ошибка времени исполнения, где-то — `undefined behavior`, а для чисел с плавающей точкой по IEEE 754 — вполне определённый результат `inf`. Один и тот же синтаксис, но разный смысл в зависимости от семантики (см. пример с эквивалентностью программ ниже).

4. Прагматика

Прагматика — это взаимодействие знаковой системы с носителем: то, как один и тот же корректный (и синтаксически, и семантически) текст программы используется, читается и интерпретируется людьми в конкретном контексте — то, что формально языком не определяется, но реально влияет на то, как код пишут и понимают.

Классический пример из естественного языка: фраза “Ты не мог бы передать соль?” семантически — это вопрос о способности, но прагматически — просьба. Смысл, который вкладывает говорящий и считывает слушающий, шире буквального значения предложения.

В коде прагматика проявляется, например, в:

- Комментариях. `// TODO`: убрать после релиза формально не существует для компилятора — лексический анализатор выбрасывает комментарий ещё на этапе токенизации, он не участвует ни в синтаксисе, ни в семантике программы. Но для человека, читающего код, это важная информация — комментарий прагматически значим, хотя семантически пуст.
- Соглашениях об именовании. `_private_field`, `CONSTANT_NAME`, `i` для счётчика цикла — язык не придаёт этим именам никакого особого смысла (это обычные идентификаторы), но программист, глядя на них, считывает намерение автора: “это приватное поле”, “это константа”, “это временная переменная”. Это тоже взаимодействие знаковой системы с носителем — соглашение, но не формальное правило языка.

Nota: Модель слабой памяти — нет гарантии, что операции в памяти в той же последовательности, что и в программе

Def. Семантика — всюду определённая функция $\llbracket \cdot \rrbracket : \mathbb{L} \mapsto \mathbb{D}$, где $\mathbb{L}$ — язык программирования, а $\mathbb{D}$ — семантический домен

Nota: Точка внутри $\llbracket \cdot \rrbracket$ — это место для подстановки программы: $\llbracket \cdot \rrbracket$ без подстановки обозначает саму функцию, а $\llbracket p \rrbracket$ — её значение на конкретной программе p . Например, для программы $p = "1 + 1"$ получаем $\llbracket p \rrbracket = 2$: сам текст программы ушёл, осталось её значение в $\mathbb{D}$.

Def. Эквивалентность программ — пусть $p_1, p_2 \in \mathbb{L}$. Тогда $p_1 \sim p_2 \Leftrightarrow \llbracket p_1 \rrbracket = \llbracket p_2 \rrbracket$

Пример. Равны ли эти программы?

```
x = y / 0
```

```
while (true) {  
    x = 3;  
}
```

В зависимости от семантики эти программы могут быть как эквивалентны, так и неэквивалентны.

Def. Трансляция — всюду определённая функция $\tau : \mathbb{L}_1 \mapsto \mathbb{L}_2$, такая что $\forall p \in \mathbb{L}_1 : \llbracket \tau(p) \rrbracket_2 = \llbracket p \rrbracket_1$. При этом обе семантики $\llbracket \cdot \rrbracket_1 : \mathbb{L}_1 \mapsto \mathbb{D}$ и $\llbracket \cdot \rrbracket_2 : \mathbb{L}_2 \mapsto \mathbb{D}$ обязаны вести в один и тот же домен $\mathbb{D}$ — иначе равенство сравнивало бы элементы разных множеств.

Пример. Пусть $\mathbb{L}_1$ — язык арифметических выражений в инфиксной записи ($2 + 3$), $\mathbb{L}_2$ — тот же язык в постфиксной записи ($2\ 3\ +$), а домен $\mathbb{D} = \mathbb{Z}$ — просто числа. Перевод τ превращает $2 + 3$ в $2\ 3\ +$. Обе семантики вычисляют выражение до числа: $\llbracket 2 + 3 \rrbracket_1 = 5$ и $\llbracket 2\ 3\ + \rrbracket_2 = 5$ — то есть $\llbracket \tau(p) \rrbracket_2 = \llbracket p \rrbracket_1 = 5$. Перевод корректен именно потому, что число на выходе одно и то же, независимо от формы записи.

Языковой процессор

Булычев говорил, что ударение в слове языковой падает на второй слог, но это неправда. На грамота.ру написано, что ударение на последний слог, а значит, правильно говорить “языковой́ процессор”.

а) Интерпретатор

Def. Интерпретатор — Пусть $\mathbb{L}$ — объектный язык с семантикой $\llbracket \cdot \rrbracket_{\mathbb{L}}$, а $\mathbb{M}$ — язык реализации с семантикой $\llbracket \cdot \rrbracket_{\mathbb{M}}$. Интерпретатор — это программа $\text{int}_{\mathbb{M}}^{\mathbb{L}} \in \mathbb{M}$, такая что $\forall p \in \mathbb{L}, x : \llbracket \text{int}_{\mathbb{M}}^{\mathbb{L}} \rrbracket_{\mathbb{M}}(p, x) = \llbracket p \rrbracket_{\mathbb{L}}(x)$

Пример. Пусть $\mathbb{L} = \text{Python}$ (объектный язык — тот, который мы интерпретируем), а $\mathbb{M} = \text{C}$ (язык реализации — на нём написан сам CPython). Тогда $\text{int}_{\mathbb{M}}^{\mathbb{L}}$ — это исходники CPython на C. Уравнение читается так: запустить CPython (как обычную программу на C) и дать ему на вход файл .py и данные — то же самое, что выполнить эту программу по правилам семантики Python.

б) Компилятор

Def. Компилятор — Пусть $\mathbb{L}$ — исходный язык, $\mathbb{L}'$ — целевой язык, оба с семантиками $\llbracket \cdot \rrbracket_{\mathbb{L}}$ и $\llbracket \cdot \rrbracket_{\mathbb{L}'}$, а $\mathbb{M}$ — язык реализации. Компилятор — это программа $\text{comp}_{\mathbb{M}}^{\mathbb{L} \rightarrow \mathbb{L}'} \in \mathbb{M}$, такая что $\forall p \in \mathbb{L}, x : \llbracket \llbracket \text{comp}_{\mathbb{M}}^{\mathbb{L} \rightarrow \mathbb{L}'} \rrbracket_{\mathbb{M}}(p) \rrbracket_{\mathbb{L}'}(x) = \llbracket p \rrbracket_{\mathbb{L}}(x)$

Пример. Пусть $\mathbb{L} = \text{C}$, $\mathbb{L}' = \text{ассемблер x86}$, а $\mathbb{M}$ — язык, на котором написан сам gcc. Тогда $\text{comp}_{\mathbb{M}}^{\mathbb{L} \rightarrow \mathbb{L}'}$ — это сам gcc как программа на языке M. Уравнение читается изнутри наружу: запустили gcc на исходнике .c $\rightarrow$ получили программу на ассемблере $\rightarrow$ выполнили её на входных данных. Результат обязан совпасть с тем, что предписывает семантика C для исходной программы.

Языки программирования делятся на высокого уровня (high-level, обозначим H) и низкого уровня (low-level, обозначим L).

Транслятор

Def. Транслятор — преобразователь языков, сохраняющий семантику.

Трансляция и транслятор — не одно и то же: трансляция это само отображение τ (математический объект), а транслятор — программа, которая это отображение реализует.

Если посмотреть, с какого уровня на какой идёт перевод, получаются четыре разных вида трансляторов:

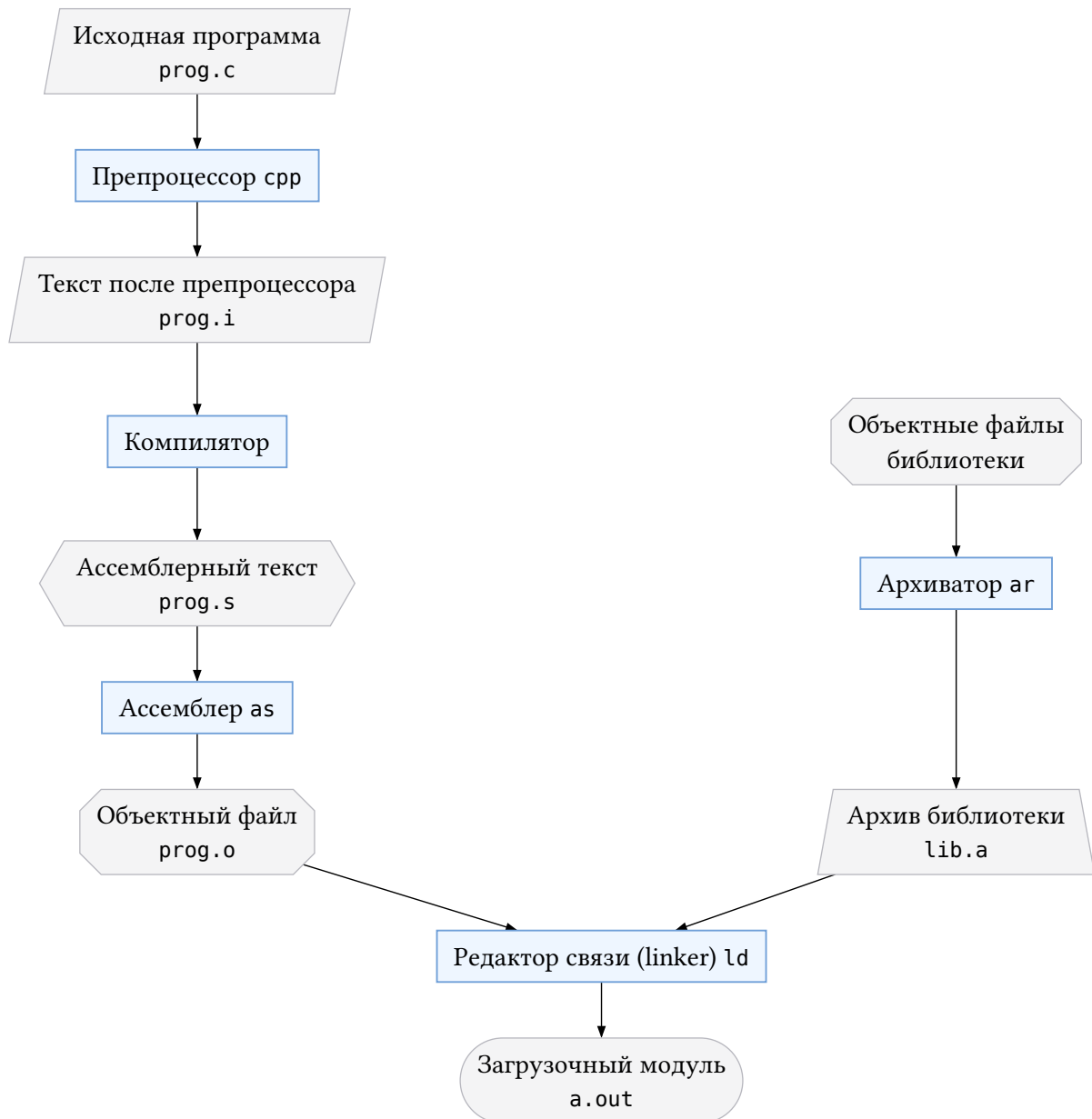
	в H	в L
из H	транспайлер / конвертор (transpiler, converter)	компилятор (compiler)
из L	декомпилятор (decompiler)	бинарный компилятор (binary compiler)

Все четыре вместе и называются трансляторами — компилятор лишь один из четырёх случаев, самый привычный.

Toolchain

Toolchain — набор инструментов, участвующих в трансляции. Их существует всего несколько десятков из-за сложности их разработки. Примеры: gcc, llvm, graalvm.

Пример: путь программы на Си от исходника до исполняемого файла.



Голубые прямоугольники — инструменты (программы, которые запускаются), серые фигуры — файлы, которые они принимают и порождают.

Nota: В toolchain входит не только эта цепочка — там ещё много дополнительных инструментов.